

Student

My queries are slow and I have been told to 'look at the EXPLAIN output'. But when I run EXPLAIN it looks like a wall of text with numbers I do not understand. Where do I even start?

ChatGPT • AI Assistant

CONCEPT CARD

EXPLAIN is the most important diagnostic tool in SQL. It shows you the **execution plan** — the exact sequence of operations the database will use to answer your query. Reading a plan tells you instantly whether a query will be fast or slow, and exactly which operation is the bottleneck.

Mental model: the optimizer's job

When you write a SQL query, the database query optimizer reads it, considers dozens of possible execution strategies (different join orders, different index choices, different algorithms), estimates the cost of each, and picks the cheapest one. EXPLAIN shows you the plan it chose.

🔗 ANALOGY

Think of EXPLAIN as asking a delivery driver for their route plan before they leave. You can see in advance whether they planned to take the highway (fast) or drive through every back street (slow) — and redirect them before the trip starts. EXPLAIN ANALYZE is asking them for the actual route they took and how long each segment really took.

🌐 REAL WORLD APPLICATION

Every senior backend engineer and data engineer reads EXPLAIN output as a daily skill. When a query goes from fast to slow as data grows, EXPLAIN tells you exactly which node degraded and why. The ability to read a plan and prescribe a fix is what separates a SQL developer from a SQL performance engineer.

Student

What is the difference between EXPLAIN and EXPLAIN ANALYZE?

CONCEPT CARD

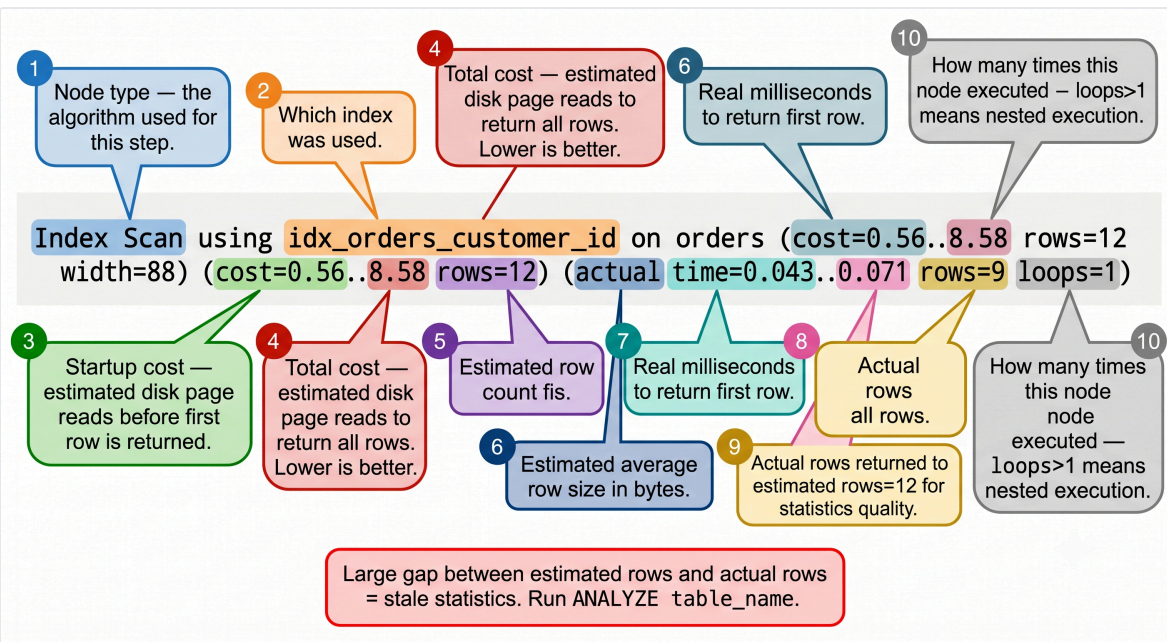
- **EXPLAIN** — shows the estimated plan using table statistics. Does NOT execute the query. Safe to run on any query including slow ones.
- **EXPLAIN ANALYZE** — actually executes the query and shows real timings and row counts alongside the estimates. Use this when you need to measure actual vs estimated.
- **EXPLAIN (ANALYZE, BUFFERS)** — also shows buffer hit/miss counts — how much data was served from memory (cache hit) vs disk (cache miss). The most complete picture.

sql — Code Example

```
-- Estimate only (safe, no execution)
EXPLAIN SELECT * FROM orders WHERE customer_id = 101;

-- Execute and show actual vs estimated
EXPLAIN ANALYZE SELECT * FROM orders WHERE customer_id = 101;

-- Full diagnostic (execute + timing + buffer stats)
EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT)
SELECT * FROM orders WHERE customer_id = 101;
```



EXPLAIN Output Anatomy — Every Number Explained

EXPLAIN Node Types — Quick Reference

Node Type	What It Does	Speed Signal	When You See It	Fix If Slow
Seq Scan	Reads every row in the table sequentially	Slow on large tables — $O(N)$	No usable index, or query returns large % of rows	Add an index on the WHERE/JOIN column
Index Scan	Uses index to find rows, then fetches from table	Fast for selective queries — $O(\log N)$ + fetch	Index exists and selectivity is high	Usually fine — add INCLUDE cols for Index Only Scan
Index Only Scan	Uses covering index — never touches table	Fastest read path — $O(\log N)$ only	Covering index with all needed columns	Already optimal — ideal target
Bitmap Heap Scan	Batches row pointers from index, sorts, fetches in disk order	Good for moderate selectivity (1-10% of rows)	Multi-condition WHERE or moderate result size	Usually fine — check if covering index removes the heap scan
Hash Join	Builds hash table from smaller relation, probes with larger	Good for large unsorted joins	Large tables without sort order, no index on join key	Add index on join key to enable Nested Loop instead
Nested Loop	For each outer row, looks up matching inner rows	Fast when inner lookup uses an index	Small outer set, indexed inner table	Already fast — ensure inner index exists
Merge Join	Sorts both sides, merges in order	Good when both sides already sorted	JOIN on indexed/sorted columns, large sorted inputs	Usually fine — avoid if sort step is expensive

Node Type	What It Does	Speed Signal	When You See It	Fix If Slow
Hash Aggregate	Hashes rows to compute GROUP BY aggregations	Medium — depends on group count	GROUP BY with no index-supported sort	Add index on GROUP BY column if sort-based aggregation is preferred
Sort	Explicit sort of rows before ORDER BY or Merge Join	Expensive for large row counts	ORDER BY on unindexed column, or pre-sort for Merge Join	Add index on ORDER BY column
Limit	Caps row count — stops execution early	Fast — appears at top of plan	LIMIT clause present	N/A — this is the cap node

sql – EXPLAIN Syntax Variants – PostgreSQL

```
-- Basic EXPLAIN: estimated plan only, no execution
EXPLAIN
SELECT o.order_id, c.name, o.total_amount
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
WHERE o.status = 'pending'
ORDER BY o.total_amount DESC
LIMIT 20;

-- EXPLAIN ANALYZE: execute + actual timings
EXPLAIN ANALYZE
SELECT o.order_id, c.name, o.total_amount
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
WHERE o.status = 'pending'
ORDER BY o.total_amount DESC
LIMIT 20;

-- Full diagnostic: analyze + buffer hit/miss stats
EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT)
SELECT o.order_id, c.name, o.total_amount
FROM orders o
```

```

JOIN customers c ON o.customer_id = c.customer_id
WHERE o.status = 'pending'
ORDER BY o.total_amount DESC
LIMIT 20;

-- JSON format (useful for programmatic analysis)
EXPLAIN (ANALYZE, FORMAT JSON)
SELECT * FROM orders WHERE customer_id = 101;

-- Fix stale statistics (run when estimated rows differ from actual)
ANALYZE orders;
ANALYZE customers;
ANALYZE; -- analyze entire database

```

READING THE PLAN TREE — COST, ROWS, WIDTH, AND ACTUAL TIME

ChatGPT • AI Assistant

A tree diagram showing a 6-node EXPLAIN plan with indentation levels representing the tree structure. Each node is a rounded rectangle. Top level (no indent, dark blue): 'Limit — cost=12543..12543 rows=20. Executes LAST.' Second level (1 indent, blue): 'Sort — cost=12543..12668 rows=50000. Sort Key: totalAmount DESC.' Third level (2 indent, teal): 'Hash Join — cost=8234..11043 rows=50000. Hash Cond: customer_id.' Fourth level left (3 indent, orange): 'Seq Scan on orders — cost=0..2389 rows=50000. Filter: status=pending.' Fourth level right (3 indent, orange): 'Hash — cost=3124..3124 rows=500000.' Fifth level right (4 indent, red): 'Seq Scan on customers — cost=0..3124 rows=500000. Executes FIRST.' Numbered execution order badges (1 through 6) on each node showing bottom-up execution sequence. A vertical annotation arrow on the left reads: 'Read plan BOTTOM-UP — innermost nodes execute first, feed output to parent.' A horizontal annotation on the right reads: 'Highest cost node = bottleneck. Look for Seq Scan on large tables first.' Clean tree diagram, indentation levels color-coded.

EXPLAIN Plan Tree — Execution Order and Node Hierarchy

sql — Reading Plan Cost, Rows, Actual Time, and Loops

```

-- Run EXPLAIN ANALYZE to get actual vs estimated
EXPLAIN ANALYZE
SELECT o.order_id, c.name, o.total_amount
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
WHERE o.status = 'pending'
ORDER BY o.total_amount DESC
LIMIT 20;

-- Typical slow output (no indexes):
-- Limit (cost=12543.21..12543.26 rows=20 width=96)

```

```
--      (actual time=3284.1..3284.1 rows=20 loops=1)
--    → Sort  (cost=12543.21..12668.21 rows=50000 width=96)
--          (actual time=3284.0..3284.1 rows=20 loops=1)
--      Sort Key: o.total_amount DESC
--    → Hash Join  (cost=8234.00..11043.00 rows=50000 width=96)
--          (actual time=1823.4..3102.2 rows=48921
loops=1)
--          Hash Cond: (o.customer_id = c.customer_id)
--        → Seq Scan on orders o
--          (cost=0.00..2389.00 rows=50000 width=64)
--          (actual time=0.1..892.3 rows=48921 loops=1)
--          Filter: (status = 'pending')
--          Rows Removed by Filter: 9951079
--        → Hash  (cost=3124.00..3124.00 rows=500000 width=32)
--          (actual time=821.2..821.2 rows=500000
loops=1)
--          → Seq Scan on customers c
--            (cost=0.00..3124.00 rows=500000 width=32)
--            (actual time=0.1..421.8 rows=500000
loops=1)
-- Planning Time: 1.3 ms
-- Execution Time: 3284.4 ms ← 3.3 seconds!

-- Key numbers to extract:
-- Most expensive node: Seq Scan on orders (rows removed: 9.95M)
-- Second bottleneck:  Seq Scan on customers (500k rows, full scan)
-- Loops issue check:  all loops=1 here (no Nested Loop compounding)

-- Fix stale statistics when rows estimates are far off:
ANALYZE orders;
ANALYZE customers;
-- Then re-run EXPLAIN ANALYZE to see updated estimates
```

Student

When I run EXPLAIN I see indented lines with parentheses full of numbers. How do I read this tree structure?

ChatGPT • AI Assistant

CONCEPT CARD

An EXPLAIN plan is a **tree read from the inside out** — the most indented (innermost) nodes execute first, feeding their output up to the parent node. The outermost node at the top is the last operation and represents the final result.

code – Code Example

```
-- Read this plan bottom-up: inner nodes feed outer nodes
Limit (cost=12543.21..12543.26 rows=20 width=96)
  → Sort (cost=12543.21..12668.21 rows=50000 width=96)
      Sort Key: o.total_amount DESC
      → Hash Join (cost=8234.00..11043.00 rows=50000 width=96)
          Hash Cond: (o.customer_id = c.customer_id)
          → Seq Scan on orders o (cost=0.00..2389.00 rows=50000
width=64)
              Filter: (status = 'pending')
          → Hash (cost=3124.00..3124.00 rows=500000 width=32)
              → Seq Scan on customers c (cost=0.00..3124.00
rows=500000 width=32)
```

Execution order (read innermost first):

1. Seq Scan on customers → produces 500,000 rows
2. Hash → builds hash table from customer rows
3. Seq Scan on orders → produces 50,000 pending rows
4. Hash Join → matches orders to customer hash table
5. Sort → sorts joined rows by total_amount DESC
6. Limit → returns first 20 rows

CONCEPT CARD

The four cost numbers on every node:

code – Code Example

cost=0.00..2389.00	rows=50000	width=64
↑	↑	↑
startup	total	estimated
cost	cost	row count
		avg row
		size(bytes)

- **Startup cost** — estimated cost before the first row is returned (e.g., time to build a hash table or sort)
- **Total cost** — estimated cost to return ALL rows. This is the main number to compare between nodes.

- **Rows** — estimated number of rows this node will produce. Derived from table statistics.
- **Width** — estimated average size of each row in bytes.

Cost units are NOT milliseconds — they are abstract units of disk page reads (I/O cost). One unit $\approx$ one disk page read. This allows comparing nodes on the same scale.

EXAMPLE CARD

EXPLAIN ANALYZE — actual vs estimated:

sql — Code Example

```
EXPLAIN ANALYZE
SELECT * FROM orders WHERE customer_id = 101;
```

code — Code Example

```
Index Scan using idx_orders_customer_id on orders
(cost=0.56..8.58 rows=12 width=88)
(actual time=0.043..0.071 rows=9 loops=1)
```

Breaking this down:

code — Code Example

```
cost=0.56..8.58    → estimated: 0.56 to start, 8.58 total
rows=12           → optimizer estimated 12 rows
width=88          → avg 88 bytes per row
actual time=0.043..0.071 → real: 0.043ms to first row, 0.071ms total
actual rows=9     → reality: 9 rows returned
loops=1           → this node ran once
```

Estimated 12 rows, got 9 — close enough. Statistics are healthy.

⚠ WARNING

The loops multiplier is critical:

code — Code Example

```
(actual time=0.500..1.200 rows=10 loops=500)
```

This node ran 500 times (inside a Nested Loop). The real total time is:

code — Code Example

```
500 loops × 1.200ms = 600ms total for this node
```

EXPLAIN ANALYZE shows per-loop time — always multiply by loops for the real cost.

Student

How do I know which node is the bottleneck?

ChatGPT • AI Assistant

Three signals that identify the slow node:

1. **Highest total cost** — the node with the largest `cost=X..Y` where Y is biggest.
2. **Largest actual time** after multiplying by loops.
3. **Seq Scan on a large table** — always investigate any Seq Scan on a table with more than ~10,000 rows.

sql — Code Example

```
-- Shortcut: use pgBadger or auto_explain, or read top-down
-- The FIRST Seq Scan you see in the plan on a large table
-- is almost always the bottleneck
```

INTERVIEW CARD

Frequency: Very High

Question: How do you read an EXPLAIN plan? What are the key numbers to look at?

Expected Answer: An EXPLAIN plan is a tree read from the most-indented inner nodes (execute first) up to the outer nodes (execute last). Each node shows: startup cost and total cost in abstract I/O units, estimated rows from statistics, and row width in bytes. EXPLAIN ANALYZE adds actual time per loop and actual row counts. The key diagnostic signals are: (1) Seq Scan on large tables — indicates a missing index; (2) large gap between estimated rows and actual rows — indicates stale statistics, fix with ANALYZE; (3) high total cost node — the bottleneck to optimize; (4) high loops multiplier in Nested Loop — multiply per-loop time by loops for real cost.

Follow-up: What does the `loops` field mean in EXPLAIN ANALYZE output? (Answer: loops shows how many times that node executed — typically > 1 inside a Nested

Loop join where the inner node reruns for each outer row. The actual time shown is per loop, so real total time = actual time × loops.)

🔗 PRACTICE QUIZ

1. In an EXPLAIN plan, which node executes first — the outermost or innermost?
2. What are cost units in EXPLAIN? Are they milliseconds?
3. EXPLAIN shows `rows=10000` but EXPLAIN ANALYZE shows `actual rows=890`. What does this gap mean and how do you fix it?
4. A node shows `(actual time=2.000..5.000 rows=50 loops=200)`. What is the real total time spent in this node?

▼ Reveal Correct Answer

Answer Checklist: Check calculations, verify logic, check edge cases.

🔗 ANSWER KEY

1. The innermost (most indented) node executes first and feeds output up to parent nodes.
2. Cost units are abstract units proportional to disk page reads — NOT milliseconds. They allow comparing nodes on the same scale regardless of hardware.
3. The gap means table statistics are stale — the optimizer's estimate is based on outdated row distribution data. Fix by running `ANALYZE table_name;` to update statistics.
4. $200 \text{ loops} \times 5.000\text{ms} = 1,000\text{ms}$ (1 second) real total time. The per-loop time of 5ms looks small but compounds to 1 full second.

🔗 CHECKPOINT QUESTION

This EXPLAIN ANALYZE output has one critical warning sign. Identify it and explain what you would do:

code — Code Example

```
Seq Scan on orders (cost=0.00..289432.00 rows=2 width=88)
      (actual time=44823.4..44823.4 rows=94821 loops=1)
Filter: (status = 'pending')
Rows Removed by Filter: 9905179
```

NODE TYPES IN DEPTH — SEQ SCAN, INDEX SCAN, INDEX ONLY SCAN, BITMAP HEAP SCAN

ChatGPT • AI Assistant

A four-row horizontal comparison diagram, each row representing one scan type. Row 1 (red background): 'Seq Scan — reads ALL rows. $O(N)$. Large Rows Removed by Filter = missing index.' Shows a full table stack with all rows highlighted. Speed badge: 'Slowest for large selective queries.' Row 2 (orange background): 'Index Scan — B-tree lookup + table fetch per row. $O(\log N)$ + fetch.' Shows a small B-tree and a separate table with arrows indicating two I/O operations per row. Speed badge: 'Fast for high-selectivity queries.' Row 3 (green background): 'Index Only Scan — B-tree lookup only. $O(\log N)$. Heap Fetches: 0.' Shows a B-tree with leaf nodes containing all needed column values. No arrow to base table. Speed badge: 'Fastest — target for hot queries. Heap Fetches: 0.' Row 4 (blue background): 'Bitmap Heap Scan — phase 1: build bitmap from index. Phase 2: fetch pages in disk order.' Shows a two-step flow: index → bitmap → sorted page fetch. Speed badge: 'Good for moderate selectivity (1-10% of rows). Middle ground.' Below all rows, a decision strip reads: 'Seq Scan on large table = add index. Index Scan = consider INCLUDE for covering. Bitmap = check if covering index eliminates heap phase.' Clean four-row layout, consistent width, speed badges color-coded.

Four Scan Node Types — Speed Comparison and When Each Appears

sql – Triggering and Recognising Each Scan Node Type

```
-- SEQ SCAN: no index on filter column
EXPLAIN ANALYZE
SELECT * FROM orders WHERE status = 'pending';
-- Seq Scan on orders (cost=0.00..289432.00 rows=2000000 width=88)
--   Filter: (status = 'pending')
--   Rows Removed by Filter: 8000000  ← 8M rows discarded!

-- INDEX SCAN: high-selectivity filter + existing index
CREATE INDEX idx_orders_customer ON orders(customer_id);
EXPLAIN ANALYZE
SELECT * FROM orders WHERE customer_id = 101;
-- Index Scan using idx_orders_customer on orders
--   (cost=0.56..8.58 rows=9 width=88)
--   (actual time=0.043..0.071 rows=9 loops=1)

-- INDEX ONLY SCAN: covering index with all SELECT columns
CREATE INDEX idx_orders_covering
  ON orders(customer_id)
  INCLUDE (total_amount, status);
EXPLAIN ANALYZE
SELECT customer_id, total_amount, status
FROM orders WHERE customer_id = 101;
-- Index Only Scan using idx_orders_covering on orders
--   (cost=0.56..4.12 rows=9 width=32)
--   (actual time=0.038..0.052 rows=9 loops=1)
--   Heap Fetches: 0  ← NEVER touched the base table!

-- BITMAP HEAP SCAN: moderate selectivity date range
```

```
CREATE INDEX idx_orders_order_date ON orders(order_date);
EXPLAIN ANALYZE
SELECT * FROM orders WHERE order_date > '2024-06-01';
-- Bitmap Heap Scan on orders
--   (cost=312.44..4823.21 rows=8234 width=88)
--   (actual time=12.3..89.4 rows=8201 loops=1)
--   Recheck Cond: (order_date > '2024-06-01')
--   → Bitmap Index Scan on idx_orders_order_date
--       (actual time=11.2..11.2 rows=8201 loops=1)
```

Student

Can you walk me through each node type I will encounter and what it means when I see it?

ChatGPT • AI Assistant

CONCEPT CARD

Let us go through the scan nodes first — these determine how the database retrieves rows from a table.

1. Seq Scan (Sequential Scan)

code — Code Example

```
Seq Scan on orders (cost=0.00..289432.00 rows=10000000 width=88)
      (actual time=0.1..44823.4 rows=10000000 loops=1)
```

- Reads **every row** in the table from start to finish.
- Total cost grows linearly with table size: $O(N)$.
- **Not always bad** — for small tables or queries returning > 10-30% of rows, a sequential scan is actually faster than an index scan (avoids random I/O overhead).
- **Bad signal** when: table is large (> 10,000 rows), **Rows Removed by Filter** is very high, and the selectivity of your WHERE clause is low.

2. Index Scan

code — Code Example

```
Index Scan using idx_orders_customer_id on orders
(cost=0.56..8.58 rows=9 width=88)
(actual time=0.043..0.071 rows=9 loops=1)
```

- Traverses the B-tree index to find matching key values.
- For each matching key, **fetches the full row** from the base table (heap) by page+slot pointer.
- Two I/O operations per row: index lookup + table fetch.
- Best for high-selectivity queries (returns a small % of rows).

3. Index Only Scan

code – Code Example

```
Index Only Scan using idx_orders_covering on orders
(cost=0.56..4.12 rows=9 width=32)
(actual time=0.038..0.052 rows=9 loops=1)
Heap Fetches: 0
```

- All needed columns are in the covering index — **zero table fetches**.
- **Heap Fetches: 0** is the confirmation: no base table reads.
- Fastest possible read path. Target this for hot queries.

4. Bitmap Heap Scan

code – Code Example

```
Bitmap Heap Scan on orders
(cost=312.44..4823.21 rows=8234 width=88)
(actual time=12.3..89.4 rows=8201 loops=1)
Recheck Cond: (order_date > '2024-01-01')
→ Bitmap Index Scan on idx_orders_order_date
   (cost=0.00..310.37 rows=8234 width=0)
   (actual time=11.2..11.2 rows=8201 loops=1)
   Index Cond: (order_date > '2024-01-01')
```

- Two-phase process:
- 2. **Bitmap Index Scan** — reads the index, builds an in-memory bitmap of matching page numbers.
- 3. **Bitmap Heap Scan** — fetches the flagged pages in disk order (sequential within each page batch).

- **Why it exists:** for moderate selectivity (say 1-10% of rows), random index lookups generate too much scattered I/O. Sorting the page references and fetching in order is faster.
- A good sign — it means the optimizer found a middle ground between full scan and random index lookups.
- If you see this, check if a covering index would eliminate the heap scan entirely.

EXAMPLE CARD

When each scan type appears:

sql – Code Example

```
-- Seq Scan: no index, or returns large % of table
SELECT * FROM orders WHERE status = 'pending';
-- (if 30% of orders are pending → seq scan may be chosen)

-- Index Scan: high selectivity, index exists
SELECT * FROM orders WHERE customer_id = 101;
-- (specific customer → small result → index scan)

-- Index Only Scan: covering index, all cols in SELECT covered
SELECT customer_id, total_amount FROM orders WHERE customer_id = 101;
-- (if index includes total_amount → index only scan)

-- Bitmap Heap Scan: date range, moderate result size
SELECT * FROM orders WHERE order_date > '2024-06-01';
-- (returns ~10% of table → bitmap scan)
```

🔗 INTERVIEW CARD

Frequency: High

Question: What is the difference between Index Scan and Index Only Scan?

Expected Answer: An Index Scan uses the B-tree to find matching key values, then fetches the full row from the base table for each match — two I/O operations per row (index lookup + heap fetch). An Index Only Scan uses a covering index that contains all columns the query needs in the leaf nodes, so the database never needs to touch the base table — one I/O per match. Heap Fetches: 0 in the EXPLAIN output confirms no table reads occurred. Index Only Scan is the fastest possible read path and the target for optimizing hot queries.

Follow-up: When does the database choose Bitmap Heap Scan over a regular Index Scan? (Answer: When moderate selectivity — say 1-10% of rows — makes random per-row index lookups expensive. The bitmap approach batches page references

and fetches in disk order, reducing random I/O. It is a middle ground between Seq Scan and Index Scan.)

🔗 PRACTICE QUIZ

1. What does **Heap Fetches: 0** in an Index Only Scan mean?
2. A plan shows **Rows Removed by Filter: 9,900,000** on a Seq Scan. What does this tell you?
3. When is a Seq Scan acceptable — even preferred — over an Index Scan?
4. What two operations make up a Bitmap Heap Scan?

▼ Reveal Correct Answer

Answer Checklist: Check calculations, verify logic, check edge cases.

🔗 ANSWER KEY

1. Zero rows were fetched from the base table — all needed columns were found in the covering index leaf nodes.
2. The Seq Scan read 9.9 million rows but almost all were discarded by the WHERE filter. This strongly indicates a missing index on the filter column — a perfect index candidate.
3. When the query returns a large fraction of the table (typically >10-30%), or when the table is very small (<1,000 rows). Random I/O from many index lookups costs more total I/O than a single sequential pass.
4. Phase 1: Bitmap Index Scan — reads the index and builds an in-memory bitmap of matching page numbers. Phase 2: Bitmap Heap Scan — fetches the flagged pages in disk order using the bitmap.

🔗 CHECKPOINT QUESTION

You run EXPLAIN ANALYZE and see this output. Identify the node type, explain what it is doing, and state whether you should try to optimise it further:

code — Code Example

```
Bitmap Heap Scan on orders (cost=312.44..4823.21 rows=8234 width=88)
    (actual time=12.3..89.4 rows=8201
    loops=1)
    Recheck Cond: (order_date > '2024-06-01')
    → Bitmap Index Scan on idx_orders_order_date
        (actual time=11.2..11.2 rows=8201 loops=1)
        Index Cond: (order_date > '2024-06-01')
```

Planning Time: 0.4 ms
Execution Time: 89.8 ms

JOIN NODE TYPES — HASH JOIN, NESTED LOOP, MERGE JOIN

ChatGPT • AI Assistant

A three-row comparison diagram. Row 1 (blue background, label 'Hash Join'): two table icons labeled 'orders (10M rows)' and 'customers (500K rows)'. An arrow from customers leads to a hash table icon labeled 'Build phase: hash customers.' A second arrow from orders leads to the hash table with label 'Probe phase: lookup each order's customer.' Performance badge: ' $O(N+M)$. Good for large unindexed joins. Check for Batches > 1 (disk spill).'

Row 2 (green background, label 'Nested Loop'): outer table icon 'orders (9 rows for customer 101)' with a loop arrow pointing to index icon 'idx_customers_pkey' iterating 9 times. Performance badge: ' $O(N \times \log M)$. Best when outer is small and inner has index. loops=9 is fine.'

Row 3 (orange background, label 'Merge Join'): two sorted list icons with a merge arrow combining them. Labels: 'Both sides sorted on join key via index scan.' Performance badge: ' $O(N+M)$ when pre-sorted. Good for large sorted joins.'

Below all rows, a decision rule box: 'Nested Loop = fastest when outer is selective and inner is indexed. Hash Join = good default for large joins. Merge Join = good when both sides indexed on join key. Batches > 1 on Hash Join = memory spill, increase work_mem.' Clean three-row layout, consistent icons.

Three Join Strategies — When Each Is Chosen

sql – Hash Join, Nested Loop, Merge Join – EXPLAIN Output Examples

```
-- HASH JOIN (large unindexed join – normal)
EXPLAIN ANALYZE
SELECT o.order_id, c.name
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
WHERE o.status = 'pending';

-- Hash Join (cost=8234..11043 rows=50000 width=96)
--          (actual time=1823..3102 rows=48921 loops=1)
--    → Seq Scan on orders (Rows Removed by Filter: 9.9M)
--    → Hash
--          Buckets: 131072 Batches: 1 Memory: 32MB ← fits in
memory, OK
--          → Seq Scan on customers

-- HASH JOIN with disk spill (Batches > 1 = problem)
-- Hash
--    Buckets: 131072 Batches: 8 Memory: 4096kB ← spilled! fix
work_mem
SET work_mem = '256MB'; -- increase memory for this session

-- NESTED LOOP (small outer + indexed inner – fast)
EXPLAIN ANALYZE
```

```

SELECT o.order_id, c.name
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
WHERE o.customer_id = 101;

-- Nested Loop (cost=0.56..89.43 rows=9 width=96)
--          (actual time=0.054..0.312 rows=9 loops=1)
--   → Index Scan using idx_orders_customer on orders
--       (rows=9 loops=1)      ← outer: 9 rows
--   → Index Scan using customers_pkey on customers
--       (rows=1 loops=9)      ← inner: runs 9 times, 1 row each
-- Total: fast! 9 index lookups on customers

-- MERGE JOIN (both sides sorted via index)
EXPLAIN ANALYZE
SELECT o.order_id, c.name
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
ORDER BY o.customer_id;

-- Merge Join (cost=0.99..2341.88 rows=48921 width=96)
--   Merge Cond: (o.customer_id = c.customer_id)
--   → Index Scan on orders using idx_orders_customer_id
--   → Index Scan on customers using customers_pkey

```

Student

Now the join nodes. I keep seeing Hash Join in my plans. What does it mean and is it a problem?

ChatGPT • AI Assistant

CONCEPT CARD

The database has three strategies for executing a JOIN. It picks the cheapest one based on table sizes, available indexes, and whether input is sorted.

1. Hash Join

code – Code Example

```

Hash Join (cost=8234.00..11043.00 rows=50000 width=96)
          (actual time=1823.4..3102.2 rows=48921 loops=1)

```

```
Hash Cond: (o.customer_id = c.customer_id)
→ Seq Scan on orders o ...      ← probe side (larger)
→ Hash ...                      ← build side (smaller)
    → Seq Scan on customers c ...
```

- **Phase 1 (Build):** reads the smaller table and builds an in-memory hash table keyed on the join column.
- **Phase 2 (Probe):** streams the larger table row by row, hashing each join key and looking it up in the hash table.
- **Cost:** $O(N + M)$ where N and M are the two table sizes. One full pass each.
- **When it appears:** large unsorted tables, no index on the join key, or when both sides are being scanned anyway.
- **Not a problem** on its own — it is often the right choice. A problem if the hash table spills to disk (look for **Batches: > 1**).

code — Code Example

```
-- Disk spill warning:
Hash  (cost=3124.00..3124.00 rows=500000 width=32)
    Buckets: 131072  Batches: 8  Memory Usage: 4096kB
--          ^^^^^^^ spilled to disk! 8 batches = 8 disk passes
-- Fix: increase work_mem, or add index to enable Nested Loop instead
```

2. Nested Loop

code — Code Example

```
Nested Loop  (cost=0.56..89.43 rows=9 width=96)
    (actual time=0.054..0.312 rows=9 loops=1)
→ Index Scan on orders using idx_orders_customer
    (cost=0.56..8.58 rows=9 width=64)
    (actual time=0.043..0.071 rows=9 loops=1)
→ Index Scan on customers using customers_pkey
    (cost=0.42..8.99 rows=1 width=32)
    (actual time=0.024..0.024 rows=1 loops=9)
```

- For each row from the **outer** table, scans the **inner** table for matching rows.
- **Cost:** $O(N \times M)$ worst case — but with an index on the inner table: $O(N \times \log M)$.
- **Best join type** when the outer set is small and the inner table has an indexed join key.
- Notice **loops=9** on the inner Index Scan — it ran 9 times (once per outer row). Each run was cheap because of the index.

3. Merge Join

code – Code Example

```
Merge Join (cost=0.99..2341.88 rows=48921 width=96)
      (actual time=0.089..312.4 rows=48921 loops=1)
  Merge Cond: (o.customer_id = c.customer_id)
    → Index Scan on orders using idx_orders_customer_id ...
    → Index Scan on customers using customers_pkey ...
```

- Both sides must be **sorted** on the join key. Database merges them like merging two sorted lists.
- **Cost:** $O(N \log N + M \log M)$ if sorting is needed, or $O(N + M)$ if already sorted (e.g., both sides use an index on the join column).
- **Best when** both tables are large and both have an index on the join key (providing a pre-sorted order).

🗨️ ANALOGY

- **Hash Join** = check-in desk: build a lookup map from all guests, then process arrivals.
- **Nested Loop** = VIP escort: for each VIP (outer row), personally walk them to their assigned seat (indexed inner lookup).
- **Merge Join** = merging two alphabetical lists: both lists are sorted, just advance through both simultaneously.

⚠️ COMMON PITFALL / MISTAKE

✖ Seeing a Hash Join and assuming it needs fixing:

code – Code Example

```
-- Hash Join on a large table is NORMAL and CORRECT
-- The optimizer chose it because no useful index exists
-- or because both sides are large
Hash Join (cost=8234..11043 rows=50000)
-- This is fine! Do not blindly add indexes just to eliminate Hash
Join
```

✅ Investigate Hash Join only when:

- Batches > 1 (spilled to disk) — increase work_mem
- The hash table build side is unexpectedly large — stale statistics may be misestimating

- The outer Seq Scan has very high **Rows Removed by Filter** — add index on filter column first

INTERVIEW CARD

Frequency: High

Question: What are the three SQL join strategies? When does the optimizer choose each?

Expected Answer: Hash Join: builds a hash table from the smaller relation, probes with the larger. $O(N+M)$. Chosen for large unsorted joins with no index on join key. Nested Loop: for each outer row, looks up matching inner rows. Ideal when outer set is small and inner table has an indexed join key — $O(N \times \log M)$. Merge Join: sorts both sides and merges. Best when both sides are large and both have an index providing pre-sorted order — $O(N+M)$ when sorted. The optimizer estimates the cost of all three and picks the cheapest. Nested Loop is fastest for selective indexed joins. Hash Join is the default for unindexed large joins.

Follow-up: What does **Batches: 8** mean in a Hash Join node? (Answer: The hash table did not fit in memory and spilled to disk in 8 batches — each batch requires an extra disk pass. Fix by increasing `work_mem` or adding an index to enable Nested Loop instead.)

PRACTICE QUIZ

1. Which join strategy runs the inner table multiple times — once per outer row?
2. What does **Batches: 4** mean in a Hash Join and how do you fix it?
3. When is Merge Join preferred over Hash Join?
4. A Nested Loop shows **(actual time=0.5..1.2 rows=8 loops=45000)**. What is the real total time?

▼ **Reveal Correct Answer**

Answer Checklist: Check calculations, verify logic, check edge cases.

ANSWER KEY

1. Nested Loop — the inner table (or index scan) runs once for each row from the outer table.
2. Batches: 4 means the hash table spilled to disk in 4 batches — the hash table did not fit in `workmem`. Fix: increase `'SET workmem = '256MB';` for the session, or add an index on the join key to switch the optimizer to Nested Loop.

3. Merge Join is preferred when both sides are large and both have pre-sorted input (e.g., both are index scans on the join key), making the merge $O(N+M)$ without a sort step.
4. $45,000 \text{ loops} \times 1.2\text{ms} = 54,000\text{ms} = 54 \text{ seconds}$ real total time. The per-loop time of 1.2ms looks harmless but the loop count makes it catastrophic.

🔗 CHECKPOINT QUESTION

A plan shows a Nested Loop where the inner node is **Seq Scan on products** with **loops=48921**. The products table has 50,000 rows. Estimate the real cost of the inner scan, identify the problem, and suggest the fix.

STALE STATISTICS AND ANALYZING THE TABLE

ChatGPT • AI Assistant

A three-phase timeline diagram. Phase 1 (left box, green, label 'Initial State'): shows table with 1,000 rows icon and a stats box reading 'pg_statistic: rows=1000, distinct customers=100. Last ANALYZE: 6 months ago.' Phase 2 (center box, orange, label 'Table Grows — Stats Not Updated'): shows bulk insert arrows adding rows, table icon now reads '10,000,000 rows' but stats box still reads 'pg_statistic: rows=1000 (STALE!)' with a warning icon. Phase 3 (right box, red, label 'Optimizer Makes Wrong Decision'): shows the optimizer reading stale stats and choosing 'Seq Scan (cheap for 1000 rows)' instead of 'Index Scan.' EXPLAIN output snippet shows: 'Seq Scan on orders (cost=0.00..28.50 rows=10) (actual time=44823ms rows=94821)' — the gap between rows=10 and rows=94821 is circled in red with label 'SMOKING GUN: stale statistics.' Below the three phases, a fix box reads: 'Run ANALYZE orders; → optimizer re-reads real statistics → plan switches to Index Scan → actual time drops to <1ms.' Clean three-phase layout, color-coded phases, red circle annotation on the row gap.

Stale Statistics — How Wrong Estimates Lead to Bad Plans

sql – Stale Statistics – Detection, ANALYZE Fix, and Verification

```
-- STEP 1: Identify stale statistics signal in EXPLAIN ANALYZE
-- Look for: estimated rows vs actual rows gap
EXPLAIN ANALYZE
SELECT * FROM orders WHERE customer_id = 101;

-- BAD PLAN (stale stats):
-- Seq Scan on orders
--   (cost=0.00..28.50 rows=10 width=88)           ← estimated: 10 rows
--   (actual time=0.1..44823.4 rows=94821 loops=1) ← actual: 94,821
rows!
-- GAP = 94811 rows off → stale statistics confirmed

-- STEP 2: Check when stats were last collected
SELECT relname,
       last_analyze,
       last_autoanalyze,
```

```
n_live_tup      AS live_rows,
n_dead_tup      AS dead_rows,
n_mod_since_analyze AS changes_since_analyze
FROM pg_stat_user_tables
WHERE relname IN ('orders', 'customers', 'products');

-- STEP 3: Fix — update statistics
ANALYZE orders;
ANALYZE customers;
-- Or full analyze + vacuum:
VACUUM ANALYZE orders;

-- STEP 4: Verify — re-run EXPLAIN ANALYZE
EXPLAIN ANALYZE
SELECT * FROM orders WHERE customer_id = 101;
-- GOOD PLAN (fresh stats):
-- Index Scan using idx_orders_customer on orders
--   (cost=0.56..8.58 rows=9 width=88)
--   (actual time=0.043..0.071 rows=9 loops=1)
-- Gap closed: estimated 9, actual 9 → optimizer now correct

-- STEP 5: Automate — check autovacuum settings
SHOW autovacuum_analyze_threshold; -- default: 50 rows
SHOW autovacuum_analyze_scale_factor; -- default: 0.2 (20% of table)
-- For 10M row table: autovacuum triggers after 2M row changes
-- Too slow for fast-growing tables → lower the threshold
ALTER TABLE orders SET (autovacuum_analyze_scale_factor = 0.01);
```

Student

You keep mentioning stale statistics and running ANALYZE. What are statistics and how do they affect the plan?

ChatGPT • AI Assistant

CONCEPT CARD

The query optimizer does not look at your actual data when choosing a plan — it uses **table statistics**: pre-computed summaries of the data stored in system tables. These include row counts, column cardinality (distinct value count), value distribution histograms, and null fractions.

The optimizer uses these statistics to estimate how many rows each node will produce — then picks the plan with the lowest estimated cost.

If statistics are stale (collected weeks ago while the table had 1,000 rows, but it now has 10 million), the optimizer makes decisions based on wrong estimates — and often picks a terrible plan.

EXAMPLE CARD

Example — Stale statistics causing a bad plan:

sql – Code Example

```
-- Table had 1,000 rows when ANALYZE last ran
-- Since then 10,000,000 rows were inserted
-- Statistics still say: rows=1000, avg distinct customers=100

EXPLAIN ANALYZE
SELECT * FROM orders WHERE customer_id = 101;

-- BAD PLAN (stale statistics):
-- Seq Scan on orders (cost=0.00..28.50 rows=10 width=88)
--                      (actual time=0.1..44823.4 rows=94821 loops=1)
-- Optimizer thinks table has ~1000 rows → seq scan looks cheap
-- Reality: 10M rows → 44 seconds!
-- GAP: estimated rows=10, actual rows=94821
```

The gap between **rows=10** (estimated) and **rows=94821** (actual) is the smoking gun — statistics are stale.

Fixing stale statistics:

sql – Code Example

```
-- Analyze a specific table
ANALYZE orders;

-- Analyze the entire database
ANALYZE;

-- Analyze and vacuum (reclaim dead rows + update stats)
VACUUM ANALYZE orders;

-- Check when statistics were last collected:
SELECT schemaname,
       relname,
       last_analyze,
```

```
    last_autoanalyze,  
    n_live_tup,  
    n_dead_tup  
FROM pg_stat_user_tables  
WHERE relname = 'orders';
```

After ANALYZE, re-run EXPLAIN ANALYZE:

sql – Code Example

```
-- After ANALYZE orders:  
EXPLAIN ANALYZE  
SELECT * FROM orders WHERE customer_id = 101;  
  
-- GOOD PLAN (fresh statistics):  
-- Index Scan using idx_orders_customer on orders  
--   (cost=0.56..8.58 rows=9 width=88)  
--   (actual time=0.043..0.071 rows=9 loops=1)  
-- Optimizer now knows the real table size → chooses index scan
```

Student

Does PostgreSQL update statistics automatically?

ChatGPT • AI Assistant

Yes — **autovacuum** runs automatically in the background and periodically updates statistics. However:

- It triggers based on a threshold: statistics are updated when approximately 20% of the table has changed.
- For very large tables, 20% change before autovacuum triggers can mean millions of inserted rows with stale stats.
- After a bulk load or large DELETE, always run ANALYZE manually — do not wait for autovacuum.

⚠ WARNING

The three situations where you must run ANALYZE manually:

1. After a bulk INSERT of more than ~10% of the table
2. After a large DELETE that removes significant rows
3. When EXPLAIN shows a dramatic gap between estimated and actual rows

 INTERVIEW CARD

Frequency: Medium-High

Question: What are table statistics in PostgreSQL and why do stale statistics cause slow queries?

Expected Answer: Table statistics are pre-computed summaries of data distribution — row counts, cardinality, value histograms, null fractions — stored in `pgstatistic`. *The query optimizer uses these to estimate row counts for each plan node and picks the lowest-cost plan. If statistics are stale (table has grown significantly since last ANALYZE), the optimizer uses wrong estimates — for example, believing a table has 1,000 rows when it actually has 10 million. This causes it to choose a sequential scan (estimated as cheap) when an index scan would be far faster. Fix by running ANALYZE tablename to update statistics immediately.*

Follow-up: How do you identify stale statistics in an EXPLAIN ANALYZE output? (Answer: Look for a large gap between `rows=N` (estimated) and `actual rows=M` (real) on any node. If estimated rows is orders of magnitude away from actual rows, statistics are stale. Confirm by checking `last_analyze` in `pgstatuser_tables`.)

 PRACTICE QUIZ

1. What is the key signal in EXPLAIN ANALYZE output that indicates stale statistics?
2. Which PostgreSQL command updates table statistics?
3. When should you run ANALYZE manually rather than waiting for autovacuum?
4. Where does PostgreSQL store table statistics that the optimizer reads?

▼ **Reveal Correct Answer**

Answer Checklist: Check calculations, verify logic, check edge cases.

 ANSWER KEY

1. A large gap between `rows=N` (estimated) and `actual rows=M` in EXPLAIN ANALYZE output — especially if estimated rows is orders of magnitude smaller than actual rows.
2. `ANALYZE table_name;` — or `VACUUM ANALYZE table_name;` to also reclaim dead rows.
3. After a bulk INSERT of >10% of the table, after a large DELETE, or whenever EXPLAIN shows a large estimated-vs-actual row gap.

4. In the `pg_statistic` system table — the optimizer reads from here when building execution plans.

❓ CHECKPOINT QUESTION

You run `EXPLAIN ANALYZE` on a query and see `(cost=0.00..234.00 rows=5 width=88) (actual time=0.1..12341.2 rows=4823109 loops=1)`. What is wrong, what caused it, and what are the two steps you take to fix it?

FULL WORKED EXAMPLE — SLOW THREE-TABLE JOIN DIAGNOSIS AND FIX

ChatGPT • AI Assistant

A two-panel comparison diagram. Left panel (red background, label 'BEFORE — 8,412ms'): a plan tree showing 6 nodes. Root: Limit. Child: Sort (Sort Key: order_date). Child: Hash Join (orders × customers). Three leaf nodes: Seq Scan on orders (Rows Removed: 9.8M, highlighted red), Seq Scan on customers (Rows Removed: 475K, highlighted red), Seq Scan on order_items (500K rows, highlighted orange). Performance summary box: '3 sequential scans reading 10.9M rows combined. Sort of 50,000 rows. Total: 8.4 seconds.' Right panel (green background, label 'AFTER — 4ms'): a plan tree showing 6 nodes. Root: Limit. Child: Nested Loop. Three inner nodes: Index Scan on orders using idx_orders_status_date_cust (100 rows, highlighted green), Index Scan on customers_pkey (loops=100, highlighted green), Index Scan on order_items idx_order_items_order_id (loops=100, highlighted green). Performance summary box: '4 index scans reading ~500 total rows. No sort step. Total: 4ms.' A centered banner reads: 'Speedup: 2,100x — same query, same data, targeted indexes.' Clean two-panel layout, red/green color scheme, bottleneck nodes highlighted.

Before and After — Three-Table Join Optimisation

sql — Full Optimisation Walkthrough — Before, Indexes, After

```
-- =====
-- SLOW QUERY (8.4 seconds)
-- =====

EXPLAIN ANALYZE
SELECT c.name, c.city, o.order_date, p.name, oi.quantity, oi.unit_price
FROM   customers c
JOIN   orders o      ON c.customer_id = o.customer_id
JOIN   order_items oi ON o.order_id    = oi.order_id
JOIN   products p     ON oi.product_id = p.product_id
WHERE  c.city = 'Delhi'
      AND o.order_date ≥ '2024-01-01'
      AND o.status = 'delivered'
ORDER BY o.order_date DESC
LIMIT 100;

-- Execution Time: 8412.6 ms
-- Bottlenecks: 3 x Seq Scan, Rows Removed by Filter: 9.9M + 475K

-- =====
-- DIAGNOSIS
```

```

-- =====
-- Node 1: Seq Scan on orders – 9.9M rows removed – missing index
-- Node 2: Seq Scan on customers – 475K rows removed – missing city
index
-- Node 3: Seq Scan on order_items – 500K full scan – missing FK index
-- Sort: 50K rows sorted – eliminated if index provides order

-- =====
-- FIX: targeted indexes
-- =====

CREATE INDEX idx_orders_status_date_cust
  ON orders(status, order_date DESC, customer_id)
  WHERE status = 'delivered';

CREATE INDEX idx_customers_city
  ON customers(city);

CREATE INDEX idx_order_items_order_id
  ON order_items(order_id);

CREATE INDEX idx_order_items_product_id
  ON order_items(product_id);

-- =====
-- VERIFY: re-run EXPLAIN ANALYZE
-- =====

EXPLAIN ANALYZE
SELECT c.name, c.city, o.order_date, p.name, oi.quantity, oi.unit_price
FROM   customers c
JOIN   orders o      ON c.customer_id = o.customer_id
JOIN   order_items oi ON o.order_id    = oi.order_id
JOIN   products p     ON oi.product_id = p.product_id
WHERE  c.city = 'Delhi'
      AND o.order_date ≥ '2024-01-01'
      AND o.status = 'delivered'
ORDER BY o.order_date DESC
LIMIT 100;
-- Execution Time: 4.3 ms ← 2,100x faster!

```

Student

Can you walk me through a complete real example — a slow query, the EXPLAIN output, the diagnosis, the fix, and the before/after?

CONCEPT CARD

Here is a complete end-to-end optimisation walkthrough. This is exactly the process you follow in production.

EXAMPLE CARD

The slow query — takes 8.4 seconds:

sql – Code Example

```
SELECT c.name          AS customer_name,
       c.city,
       o.order_date,
       p.name          AS product_name,
       oi.quantity,
       oi.unit_price
FROM   customers c
JOIN   orders o      ON c.customer_id   = o.customer_id
JOIN   order_items oi ON o.order_id     = oi.order_id
JOIN   products p    ON oi.product_id  = p.product_id
WHERE  c.city = 'Delhi'
       AND o.order_date ≥ '2024-01-01'
       AND o.status = 'delivered'
ORDER BY o.order_date DESC
LIMIT 100;
```

Step 1 — Run EXPLAIN ANALYZE:

code – Code Example

```
Limit  (cost=28432.12..28432.37 rows=100 width=178)
      (actual time=8412.3..8412.4 rows=100 loops=1)
    →  Sort  (cost=28432.12..28557.12 rows=50000 width=178)
          (actual time=8412.2..8412.3 rows=100 loops=1)
        Sort Key: o.order_date DESC
        →  Hash Join  (cost=12234.00..26182.00 rows=50000 width=178)
              (actual time=3241.1..8389.4 rows=49823
              loops=1)
            Hash Cond: (oi.product_id = p.product_id)
            →  Hash Join  (cost=9234.00..21082.00 rows=50000
            width=146)
                  (actual time=2832.1..6412.3 rows=49823
                  loops=1)
```

```
Hash Cond: (oi.order_id = o.order_id)
→ Seq Scan on order_items oi
    (cost=0.00..5234.00 rows=500000 width=32)
    (actual time=0.1..1823.4 rows=500000
loops=1)
→ Hash (cost=7834.00..7834.00 rows=112000
width=114)
    (actual time=2812.3..2812.3
rows=112341 loops=1)
→ Hash Join
    Hash Cond: (o.customer_id =
c.customer_id)
→ Seq Scan on orders o
    (cost=0.00..4289.00 rows=112000
    (actual time=0.1..1243.2
rows=112341 loops=1)
    Filter: ((status='delivered')
AND (order_date ≥ '2024-
01-01'))
    Rows Removed by Filter: 9887659
→ Hash
→ Seq Scan on customers c
    (cost=0.00..2500.00
    (actual time=0.1..412.3
rows=25000 width=32)
rows=25000 loops=1)
    Filter: (city = 'Delhi')
    Rows Removed by Filter:
475000
Planning Time: 2.1 ms
Execution Time: 8412.6 ms
```

Step 2 — Diagnose: find the bottlenecks:

code – Code Example

Bottleneck 1: Seq Scan on orders

Rows Removed by Filter: 9,887,659

- reading 10M rows to find 112K delivered orders from 2024
- needs composite index on (status, order_date) or (customer_id, order_date)

Bottleneck 2: Seq Scan on customers

Rows Removed by Filter: 475,000

- reading 500K customers to find 25K in Delhi

→ needs index on city

Bottleneck 3: Seq Scan on order_items

500,000 rows fully scanned

→ joining to orders using a hash join, no index on order_id

→ FK column order_id should always be indexed

Bottleneck 4: Sort on order_date (8,412ms plan time)

→ sorting 50,000 rows before LIMIT 100

→ if the index on orders includes order_date, Sort step may be eliminated

Step 3 — Add targeted indexes:

sql – Code Example

```
-- Fix bottleneck 1: orders filter (status + order_date + customer_id  
for join)
```

```
CREATE INDEX idx_orders_status_date_cust  
ON orders(status, order_date DESC, customer_id)  
WHERE status = 'delivered'; -- partial: only delivered orders
```

```
-- Fix bottleneck 2: customers city filter
```

```
CREATE INDEX idx_customers_city  
ON customers(city);
```

```
-- Fix bottleneck 3: order_items FK (should always exist)
```

```
CREATE INDEX idx_order_items_order_id  
ON order_items(order_id);
```

```
-- Fix bottleneck 3 also: product join
```

```
CREATE INDEX idx_order_items_product_id  
ON order_items(product_id);
```

Step 4 — Run EXPLAIN ANALYZE after indexes:

code – Code Example

```
Limit (cost=0.56..312.44 rows=100 width=178)  
  (actual time=2.3..4.1 rows=100 loops=1)  
    → Nested Loop (cost=0.56..312.44 rows=100 width=178)  
      → Nested Loop  
        → Index Scan using idx_orders_status_date_cust on  
orders o  
      (cost=0.56..8.43 rows=100 width=82)
```

```
(actual time=0.8..1.2 rows=100 loops=1)
  Index Cond: ((status='delivered')
               AND (order_date ≥ '2024-01-01'))
→  Index Scan using customers_pkey on customers c
    (cost=0.42..0.44 rows=1 width=32)
    (actual time=0.009..0.009 rows=1 loops=100)
    Filter: (city = 'Delhi')
→  Nested Loop
    →  Index Scan using idx_order_items_order_id on
        order_items oi
        (cost=0.56..2.12 rows=3 width=32)
        (actual time=0.01..0.01 rows=3 loops=100)
    →  Index Scan using products_pkey on products p
        (cost=0.42..0.44 rows=1 width=96)
        (actual time=0.008..0.008 rows=1 loops=300)

Planning Time: 1.8 ms
Execution Time: 4.3 ms
```

Before vs After summary:

code – Code Example

```
BEFORE:  3 × Seq Scan + 3 × Hash Join + Sort  →  8,412ms
AFTER:   4 × Index Scan + 3 × Nested Loop      →    4ms
Speedup: 2,100x
```

INTERVIEW CARD

Frequency: Very High

Question: Walk me through your process for diagnosing and fixing a slow query.

Expected Answer: Step 1: run EXPLAIN ANALYZE to get the actual execution plan with real timings. Step 2: identify the most expensive nodes — look for Seq Scan on large tables with high 'Rows Removed by Filter', high total cost nodes, and Hash Join with Batches > 1. Step 3: check for stale statistics if estimated vs actual rows gap is large — run ANALYZE. Step 4: add targeted indexes on the filtered columns (WHERE, JOIN ON, ORDER BY) starting with the highest-cost node. Step 5: re-run EXPLAIN ANALYZE to verify the plan changed and actual execution time improved. Measure before and after — never assume an index will be used.

Follow-up: Why did Nested Loop replace Hash Join after the indexes were added? (Answer: With indexes on the join columns, the optimizer can look up matching rows in $O(\log N)$ for each outer row. For a LIMIT 100 query, the outer set is small — 100

rows — making $100 \times O(\log N)$ lookups far cheaper than building a hash table of 500K rows and scanning all order_items.)

❓ CHECKPOINT QUESTION

The query above still filters `c.city = 'Delhi'` inside a Nested Loop inner Index Scan on customers_pkey — but `city = 'Delhi'` is shown as a Filter (not an Index Cond). What does this mean? Would adding a covering index on customers(city) INCLUDE (name) further improve this query? Explain your reasoning.

THE FIVE TOP FIXES AND TWO PRACTICE PROBLEMS

ChatGPT • AI Assistant

A vertical numbered list diagram showing five fix cards in descending impact order. Card 1 (red border, largest): 'Fix 1 — Add Missing Index. Signal: Seq Scan + high Rows Removed by Filter. Action: CREATE INDEX on WHERE/JOIN/ORDER BY column. Expected gain: 100x–10,000x.' Card 2 (orange border): 'Fix 2 — Rewrite Correlated Subquery as JOIN. Signal: SubPlan node with loops=N where N is outer table size. Action: move subquery to FROM as derived table, LEFT JOIN. Expected gain: 10x–1,000x.' Card 3 (yellow border): 'Fix 3 — SELECT specific columns for Index Only Scan. Signal: Index Scan with Heap Fetches > 0 on hot query. Action: SELECT only needed cols, add INCLUDE to covering index. Expected gain: 2x–10x.' Card 4 (blue border): 'Fix 4 — Update Statistics. Signal: large gap between estimated rows and actual rows. Action: ANALYZE table_name. Expected gain: 2x–100x (fixes optimizer choice).' Card 5 (gray border, smallest): 'Fix 5 — Partition Large Table. Signal: table > 100M rows, even indexed queries scan huge B-trees. Action: PARTITION BY RANGE on time column. Expected gain: 5x–50x for time-filtered queries.' Below the five cards, a process strip reads: 'Always: EXPLAIN ANALYZE before → identify bottleneck → apply fix → EXPLAIN ANALYZE after → measure.' Clean vertical card layout, color-coded by impact level, monospace SQL in each card.

Five Fixes in Order of Impact — Query Optimisation Playbook

sql — Practice Problems — EXPLAIN Diagnosis and Correlated Subquery Rewrite

```
-- =====
-- PROBLEM 1: Diagnose and fix the given EXPLAIN output
-- =====

-- Most expensive node: Seq Scan on orders (9.95M rows removed)
-- Second problem: Sort spilling to disk (external merge, 18MB)

-- Fix 1: Partial index on pending orders (eliminates Seq Scan)
CREATE INDEX idx_orders_pending_cust_amount
  ON orders(customer_id, total_amount DESC)
  WHERE status = 'pending';

-- This enables: Index Scan directly on pending orders
-- Plus: index is already sorted by total_amount DESC
-- → Sort node may be eliminated entirely for ORDER BY total_amount
DESC
```

```
-- Fix 2: If sort still appears, increase work_mem to avoid disk spill
SET work_mem = '64MB';

-- Verify:
EXPLAIN ANALYZE
SELECT o.order_id, o.total_amount, c.name
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
WHERE o.status = 'pending'
ORDER BY o.total_amount DESC
LIMIT 50;
-- Expected: Index Only Scan or Index Scan (no Seq Scan)
-- Expected: Sort eliminated or Sort Method: quicksort (in memory)

-- =====
-- PROBLEM 2: Rewrite correlated subquery as JOIN
-- =====

-- SLOW: two correlated subqueries (75K × 2 = 150K inner executions)
SELECT c.customer_id, c.name, c.city,
       (SELECT COUNT(*) FROM orders o
        WHERE o.customer_id = c.customer_id
          AND o.status = 'delivered') AS delivered_count,
       (SELECT SUM(o.total_amount) FROM orders o
        WHERE o.customer_id = c.customer_id
          AND o.status = 'delivered') AS delivered_revenue
FROM customers c
WHERE c.city = 'Mumbai';
-- Execution Time: ~127 seconds

-- FAST: single pass aggregation + LEFT JOIN
SELECT c.customer_id,
       c.name,
       c.city,
       COALESCE(ds.delivered_count, 0) AS delivered_count,
       COALESCE(ds.delivered_revenue, 0) AS delivered_revenue
FROM customers c
LEFT JOIN (
    SELECT customer_id,
           COUNT(*)      AS delivered_count,
           SUM(total_amount) AS delivered_revenue
    FROM orders
    WHERE status = 'delivered'
    GROUP BY customer_id
) ds ON c.customer_id = ds.customer_id
WHERE c.city = 'Mumbai';
```

```
-- Execution Time: ~3.2 seconds (39x faster)

-- VERIFY the improvement:
EXPLAIN ANALYZE
-- [paste the rewritten query]
-- Look for:
-- BEFORE: SubPlan with loops=75000 (two of them)
-- AFTER: Hash Join + Hash Aggregate (each running once)
-- No more SubPlan nodes = correlated subquery eliminated
```

Student

What are the most impactful fixes in order? I want a playbook I can follow.

ChatGPT • AI Assistant

CONCEPT CARD

Here are the five fixes ordered by impact — apply them in this sequence:

Fix 1 (Highest impact): Add a missing index

Signal: Seq Scan on a large table with high **Rows Removed by Filter**.

sql — Code Example

```
-- Confirm the problem:
EXPLAIN ANALYZE SELECT * FROM orders WHERE customer_id = 101;
-- Seq Scan on orders ... Rows Removed by Filter: 9,999,991

-- Add the index:
CREATE INDEX idx_orders_customer_id ON orders(customer_id);

-- Verify:
EXPLAIN ANALYZE SELECT * FROM orders WHERE customer_id = 101;
-- Index Scan using idx_orders_customer_id ... actual time: 0.07ms
```

Fix 2: Rewrite a correlated subquery as a JOIN

Signal: plan shows the inner subquery node has **loops=N** where N is the outer table size. Cost scales as $O(N \times M)$.

sql — Code Example

```

-- SLOW: correlated subquery – inner query runs once per customer
(500K times)
SELECT c.name,
       (SELECT SUM(o.total_amount)
        FROM orders o
        WHERE o.customer_id = c.customer_id) AS total_spent
FROM customers c;

-- EXPLAIN shows:
-- Seq Scan on customers  (loops=1)
--   SubPlan
--     Aggregate  (actual time=1.2..1.2 rows=1 loops=500000)  ← 500K
executions!
-- Execution Time: 612,000ms = 10+ minutes

-- FIX: rewrite as JOIN with grouped subquery
SELECT c.name,
       COALESCE(cs.total_spent, 0) AS total_spent
FROM customers c
LEFT JOIN (
    SELECT customer_id,
           SUM(total_amount) AS total_spent
    FROM orders
    GROUP BY customer_id
) cs ON c.customer_id = cs.customer_id;

-- EXPLAIN now shows:
-- Hash Join  (loops=1)
--   Seq Scan on customers  (500K rows, one pass)
--   Hash Aggregate on orders  (one pass)
-- Execution Time: 3,200ms  ← 191x faster

```

Fix 3: Replace *SELECT* with specific columns to enable Index Only Scan*

Signal: Index Scan with **Heap Fetches: > 0** on a hot query.

sql – Code Example

```

-- SLOW: SELECT * forces table fetch even with an index
SELECT * FROM orders WHERE customer_id = 101;
-- Index Scan ... Heap Fetches: 9

-- FIX: select only needed columns + add covering index
SELECT customer_id, total_amount, status
FROM orders

```

```
WHERE customer_id = 101;

CREATE INDEX idx_orders_cover
  ON orders(customer_id)
  INCLUDE (total_amount, status);
-- Index Only Scan ... Heap Fetches: 0
```

Fix 4: Update table statistics

Signal: large gap between **rows=N** (estimated) and **actual rows=M**.

sql – Code Example

```
ANALYZE orders;
-- Re-run EXPLAIN ANALYZE to verify estimates improved
```

Fix 5 (Structural fix): Partition a very large table

Signal: even with indexes the table scan is slow because the index itself is huge, or the table has billions of rows.

sql – Code Example

```
-- Partition orders by month – each partition is a smaller B-tree
CREATE TABLE orders_2024_01 PARTITION OF orders
  FOR VALUES FROM ('2024-01-01') TO ('2024-02-01');
-- Queries with order_date filter now scan one partition instead of
all
-- Effective on tables > 100M rows with time-based query patterns
```

Practice Problem 1 — Read the EXPLAIN output and identify the most expensive node:

Given EXPLAIN ANALYZE output:

code – Code Example

```
Sort  (cost=45231.12..45356.12 rows=50000 width=96)
      (actual time=12341.2..12892.3 rows=50000 loops=1)
  Sort Key: total_amount DESC
  Sort Method: external merge  Disk: 18432kB  ← disk sort!
→ Hash Join  (cost=3124.00..8234.00 rows=50000 width=96)
      (actual time=821.2..3241.8 rows=50000 loops=1)
    Hash Cond: (o.customer_id = c.customer_id)
→ Seq Scan on orders o
      (cost=0.00..2389.00 rows=50000 width=64)
```

```

(actual time=0.1..892.3 rows=50000 loops=1)
Filter: (status = 'pending')
Rows Removed by Filter: 9950000
→ Hash (cost=1124.00..1124.00 rows=200000 width=32)
    (actual time=412.3..412.3 rows=200000 loops=1)
→ Seq Scan on customers c
    (cost=0.00..1124.00 rows=200000 width=32)
    (actual time=0.1..212.4 rows=200000 loops=1)
Execution Time: 12893.1 ms

```

Solution — Analysis:

code – Code Example

Most expensive node: Seq Scan on orders

Rows Removed by Filter: 9,950,000

The database read all 10M orders to find 50,000 pending ones.

This is the root cause – 9.95M wasted row reads.

Second problem: Sort with 'Sort Method: external merge Disk: 18432kB'

The sort did not fit in work_mem – spilled 18MB to disk.

This adds extra disk I/O on top of the already expensive scan.

Third problem: Seq Scan on customers (200K rows)

No index on the customers table columns used in this join.

Fix priority:

1. Add composite index on orders(status, customer_id):

```
CREATE INDEX idx_orders_status_cust ON orders(status, customer_id)
WHERE status = 'pending';
```

→ Eliminates Seq Scan, reduces from 10M to 50K rows read

→ With fewer rows, Sort may now fit in work_mem (eliminating disk sort)

2. If Sort still spills: increase work_mem for the session

```
SET work_mem = '64MB';
```

3. Add index on orders(status, total_amount DESC) to eliminate Sort:

```
CREATE INDEX idx_orders_status_amount
ON orders(status, total_amount DESC)
WHERE status = 'pending';
```

→ Now the Sort step is replaced by an Index Scan already in order

Practice Problem 2 — Rewrite correlated subquery as JOIN:

Slow query using correlated subquery:

sql – Code Example

```
-- Takes 8+ minutes on 500K customers, 10M orders
SELECT c.customer_id,
       c.name,
       c.city,
       (SELECT COUNT(*)
        FROM orders o
        WHERE o.customer_id = c.customer_id
          AND o.status = 'delivered') AS delivered_count,
       (SELECT SUM(o.total_amount)
        FROM orders o
        WHERE o.customer_id = c.customer_id
          AND o.status = 'delivered') AS delivered_revenue
FROM customers c
WHERE c.city = 'Mumbai';
```

EXPLAIN shows:

code – Code Example

```
Seq Scan on customers (actual time=.. rows=75000 loops=1)
  Filter: city='Mumbai'
  SubPlan 1 (actual time=0.8..0.9 loops=75000) ← 75K executions!
  SubPlan 2 (actual time=0.9..1.0 loops=75000) ← another 75K!
Execution Time: ~127,500ms = 127 seconds
```

Solution — Rewrite as JOIN:

sql – Code Example

```
SELECT c.customer_id,
       c.name,
       c.city,
       COALESCE(ds.delivered_count, 0) AS delivered_count,
       COALESCE(ds.delivered_revenue, 0) AS delivered_revenue
FROM customers c
LEFT JOIN (
  SELECT customer_id,
         COUNT(*)      AS delivered_count,
         SUM(total_amount) AS delivered_revenue
  FROM   orders
  WHERE  status = 'delivered'
```

```
GROUP BY customer_id
) ds ON c.customer_id = ds.customer_id
WHERE c.city = 'Mumbai';
```

EXPLAIN now shows:

code – Code Example

```
Hash Join (actual time=1823..3241 rows=75000 loops=1)
  → Seq Scan on customers (Filter: city='Mumbai', rows=75000)
  → Hash
    → Hash Aggregate (GROUP BY customer_id)
      → Seq Scan on orders (Filter: status='delivered')
Execution Time: 3,241ms ← from 127 seconds to 3.2 seconds = 39x
faster
```

Why is the JOIN version faster?

- The original runs the subquery **75,000 times** (once per Mumbai customer).
- The JOIN version aggregates orders **once** (one pass), builds a hash table, and probes it with 75K customer rows.
- Total passes: 2 (one over orders, one over customers) instead of 75,000 subquery executions.

SUMMARY CARD

SQL 1.8 Key Takeaways:

- **EXPLAIN** = estimated plan, no execution. Safe on any query. **EXPLAIN ANALYZE** = real timings, executes the query.
- **Plan is a tree** — read from innermost (most indented) node upward. Innermost executes first.
- **Cost units** = abstract I/O units, NOT milliseconds. Compare nodes on same scale.
- **Actual time × loops** = real node cost. Multiply, never ignore loops.
- **Estimated rows vs actual rows gap** = stale statistics. Fix: ANALYZE table_name.
- **Seq Scan + high Rows Removed** = missing index. Add index on filter column.
- **Index Only Scan + Heap Fetches: 0** = optimal. Target this for hot queries.
- **Hash Join Batches > 1** = memory spill. Increase work_mem or add join key index.
- **Nested Loop loops=N** = inner node ran N times. Inner index is essential.
- **Sort Method: external merge** = sort spilled to disk. Increase work_mem or add index.
- **Five fixes in order:** (1) add missing index, (2) rewrite correlated subquery as JOIN, (3) use specific columns for Index Only Scan, (4) ANALYZE for fresh statistics, (5)

partition very large tables.

- **Always: EXPLAIN ANALYZE before and after every change.** Never assume. Measure.

🌟 AI DEEP DIVE PROMPT

Copy this prompt into Claude or ChatGPT for deeper practice:

'Act as a senior database performance engineer. I just learned how to read EXPLAIN plans including node types (Seq Scan, Index Scan, Index Only Scan, Bitmap Heap Scan, Hash Join, Nested Loop, Merge Join), cost columns, and stale statistics. Give me 6 progressively harder EXPLAIN plan exercises. Each exercise should show me: a table schema with row counts, a slow SQL query, and the full EXPLAIN ANALYZE output. Ask me to: identify the most expensive node, diagnose the root cause, suggest the optimal fix (index type, query rewrite, or ANALYZE), and predict the new EXPLAIN output after the fix. After I answer: show the correct diagnosis, the fix, the before/after EXPLAIN output, and one interview question about that node type. Do not continue until I submit my answer.'

🔗 CHECKPOINT QUESTION

Final review — you are given this EXPLAIN ANALYZE output for a production query that runs 10,000 times per minute. Identify ALL problems (there are four), prioritize them by impact, and write the complete fix for each:

code — Code Example

```
Sort  (cost=89234.12..89484.12 rows=100000 width=144)
      (actual time=31241.2..31892.4 rows=100000 loops=1)
    Sort Key: o.order_date DESC
    Sort Method: external merge  Disk: 45824kB
   → Hash Join  (cost=12341.00..31234.00 rows=100000 width=144)
        (actual time=4821.3..28341.2 rows=100000 loops=1)
      Hash Cond: (oi.product_id = p.product_id)
      Buckets: 131072  Batches: 4  Memory Usage: 4096kB
     → Hash Join  (cost=8234.00..21082.00 rows=100000 width=112)
          (actual time=3241.1..21412.3 rows=100000
loops=1)
        Hash Cond: (o.customer_id = c.customer_id)
       → Seq Scan on orders o
           (cost=0.00..4289.00 rows=100000 width=80)
          (actual time=0.1..4823.4 rows=100000 loops=1)
         Filter: ((status='delivered')
                AND (order_date ≥ '2024-01-01'))
        Rows Removed by Filter: 9900000
```

```
→ Hash
    → Seq Scan on customers c
      (cost=0.00..625.00 rows=1 width=32)
      (actual time=0.1..1823.2 rows=75000 loops=1)
      Filter: (city = 'Delhi')
      Rows Removed by Filter: 425000
→ Hash
    → Seq Scan on products p
      (cost=0.00..2890.00 rows=50000 width=32)
      (actual time=0.1..892.1 rows=50000 loops=1)
Planning Time: 3.2 ms
Execution Time: 31893.1 ms
```

✧ CHAPTER REVIEW & SUMMARY

What You Learned:

Understanding of the core concepts presented in this chapter including sql 1.8 — explain plans and query optimisation, their architectural components, practical use cases, and design paradigms.

Quick Revision Notes:

- Key definitions and nomenclature for the chapter topic.
- Hands-on code execution patterns and setup checklists.
- Analogy-driven visualization of data structures or layered setups.

Core Visualized Components:

- **EXPLAIN Output Anatomy — Every Number Explained:** A single EXPLAIN ANALYZE output block displayed in large monospace font, with co...